MoneyPoly White-Box Testing and Code Quality Report

Contents

1 White-Box Test Cases Report **2**
1.1 Scope 2
1.2 Individual Test Case Details 2

2 Detailed Test Cases **2**

3 Pylint Code Quality Report **14**

1 White-Box Test Cases Report

1.1 Scope

This report is for branch-focused white-box testing of MoneyPoly.

Files tested:

- moneypoly/bank.py
- moneypoly/board.py
- moneypoly/cards.py
- moneypoly/player.py
- moneypoly/property.py
- moneypoly/game.py

Run command:

```
python -m unittest tests.test_white_box -v
```

1.2 Individual Test Case Details

Format used:

- Why needed
- What it is testing
- Expected result
- Actual result

2 Detailed Test Cases

```
# 1.3 White-Box Test Cases Report

## Scope
This report is for branch-focused white-box testing of MoneyPoly.

Files tested:
- moneypoly/bank.py
- moneypoly/board.py
- moneypoly/cards.py
- moneypoly/player.py
- moneypoly/property.py
- moneypoly/game.py

Run command:
```

```bash
python -m unittest tests.test_white_box -v
```

Individual Test Case Details

Format used for each test:
- Why needed
- What it is testing
- Expected result
- Actual result

A) Bank Test Cases

1. test_collect_negative_amount_is_ignored
- Why needed: validates negative-input guard branch.
- What it is testing: collect(amount) with amount < 0.
- Expected result: bank balance stays unchanged.
- Actual result: PASS.

2. test_collect_positive_amount_is_added
- Why needed: validates normal collection path.
- What it is testing: collect(amount) with positive value.
- Expected result: bank balance increases by amount.
- Actual result: PASS.

3. test_pay_out_negative_amount_returns_zero
- Why needed: validates early-return branch for invalid payout amount.
- What it is testing: pay_out(amount) when amount <= 0.
- Expected result: return 0 and no balance change.
- Actual result: PASS.

4. test_pay_out_normal_amount_reduces_funds
- Why needed: validates normal payout behavior.
- What it is testing: pay_out(amount) with valid amount within funds.
- Expected result: returned amount equals requested amount and funds decrease.
- Actual result: PASS.

5. test_pay_out_raises_when_amount_exceeds_funds
- Why needed: validates insufficient-funds error branch.
- What it is testing: pay_out(amount) when amount > current funds.
- Expected result: ValueError is raised.
- Actual result: PASS.

6. test_give_loan_negative_amount_is_ignored
- Why needed: validates guard branch for invalid loan amount.
- What it is testing: give_loan(player, amount) with amount <= 0.
- Expected result: no changes to bank funds, player balance, or loan count.

- Actual result: PASS.

7. test_give_loan_normal_amount_updates_player_and_bank
- Why needed: validates normal integration path of loan disbursement.
- What it is testing: give_loan(player, amount) with positive amount.
- Expected result: bank funds decrease, player balance increases, loan log updates.
- Actual result: PASS.

B) Player Test Cases

8. test_move_awards_salary_when_passing_go
- Why needed: validates wrap-around branch on board movement.
- What it is testing: move(steps) crossing board end.
- Expected result: position wraps and GO salary is added.
- Actual result: PASS.

9. test_move_without_wrap_does_not_award_salary
- Why needed: validates non-wrap branch of movement.
- What it is testing: move(steps) without crossing board end.
- Expected result: position advances normally with no salary.
- Actual result: PASS.

10. test_add_money_rejects_negative_amount
- Why needed: validates invalid-input branch.
- What it is testing: add_money(amount) with negative value.
- Expected result: ValueError is raised.
- Actual result: PASS.

11. test_net_worth_includes_property_values
- Why needed: winner logic depends on correct net worth.
- What it is testing: net_worth() with owned property.
- Expected result: net worth equals balance + property prices.
- Actual result: PASS.

C) Property Test Cases

12. test_full_group_ownership_doubles_rent
- Why needed: validates monopoly/full-group rent branch.
- What it is testing: get_rent() when all group properties owned by same player.
- Expected result: rent is doubled.
- Actual result: PASS.

13. test_mortgaged_property_has_zero_rent
- Why needed: validates mortgage guard branch.
- What it is testing: get_rent() when property is mortgaged.
- Expected result: rent is 0.
- Actual result: PASS.

D) Board Test Cases

14. test_get_property_at_returns_property_when_found
- Why needed: validates successful property lookup path.
- What it is testing: get_property_at(position) for known property tile.
- Expected result: Property object is returned.
- Actual result: PASS.

15. test_get_property_at_returns_none_when_missing
- Why needed: validates missing-property branch.
- What it is testing: get_property_at(position) for non-property tile.
- Expected result: None is returned.
- Actual result: PASS.

16. test_get_tile_type_returns_special_for_special_tile
- Why needed: validates special-tile decision branch.
- What it is testing: get_tile_type(position) for GO.
- Expected result: returns special tile label.
- Actual result: PASS.

17. test_get_tile_type_returns_property_for_property_tile
- Why needed: validates property-tile branch.
- What it is testing: get_tile_type(position) for a property position.
- Expected result: returns property.
- Actual result: PASS.

18. test_get_tile_type_returns_blank_for_unknown_tile
- Why needed: validates fallback branch.
- What it is testing: get_tile_type(position) for non-special, non-property tile.
- Expected result: returns blank.
- Actual result: PASS.

19. test_is_purchasable_false_when_position_has_no_property
- Why needed: validates no-property branch in purchasable check.
- What it is testing: is_purchasable(position) on special tile.
- Expected result: returns False.
- Actual result: PASS.

20. test_is_purchasable_false_when_property_is_mortgaged
- Why needed: validates mortgaged branch.
- What it is testing: is_purchasable(position) for mortgaged property.
- Expected result: returns False.
- Actual result: PASS.

21. test_is_purchasable_false_when_property_is_owned
- Why needed: validates owned-property branch.
- What it is testing: is_purchasable(position) for owned property.
- Expected result: returns False.

- Actual result: PASS.

22. test_is_purchasable_true_when_unowned_and_not_mortgaged
- Why needed: validates normal purchasable path.
- What it is testing: is_purchasable(position) for available property.
- Expected result: returns True.
- Actual result: PASS.

23. test_is_special_tile_true_and_false_paths
- Why needed: validates both boolean branches.
- What it is testing: is_special_tile(position) on special and non-special positions.
- Expected result: True for special tile, False otherwise.
- Actual result: PASS.

E) CardDeck Test Cases

24. test_draw_returns_none_when_deck_empty
- Why needed: validates empty-deck branch.
- What it is testing: draw() on empty deck.
- Expected result: returns None.
- Actual result: PASS.

25. test_draw_cycles_cards_in_order
- Why needed: validates normal and cycling behavior.
- What it is testing: draw() sequence beyond deck length.
- Expected result: cards repeat from start after last card.
- Actual result: PASS.

26. test_peek_returns_none_when_deck_empty
- Why needed: validates empty-deck peek branch.
- What it is testing: peek() on empty deck.
- Expected result: returns None.
- Actual result: PASS.

27. test_peek_does_not_advance_index
- Why needed: validates peek non-mutating behavior.
- What it is testing: peek() followed by draw().
- Expected result: draw() returns same next card seen by peek().
- Actual result: PASS.

28. test_cards_remaining_reports_expected_count_before_cycle
- Why needed: validates remaining-count logic path.
- What it is testing: cards_remaining() before and after draw().
- Expected result: count decreases by one after draw.
- Actual result: PASS.

F) Game Test Cases

29. test_buy_property_fails_when_insufficient_balance
- Why needed: validates insufficient-balance branch.
- What it is testing: buy_property() with balance < price.
- Expected result: returns False and owner stays None.
- Actual result: PASS.

30. test_play_turn_jailed_player_path_advances_turn
- Why needed: validates jailed-player branch in turn loop.
- What it is testing: play_turn() when current player is in jail.
- Expected result: jail handler called and turn advances.
- Actual result: PASS.

31. test_play_turn_three_doubles_branch_sends_player_to_jail
- Why needed: validates three-doubles branch.
- What it is testing: play_turn() when doubles streak >= 3.
- Expected result: player sent to jail, movement skipped, turn advances.
- Actual result: PASS.

32. test_play_turn_doubles_branch_keeps_same_player
- Why needed: validates doubles extra-turn branch.
- What it is testing: play_turn() when is_doubles() is True.
- Expected result: move resolves, turn does not advance.
- Actual result: PASS.

33. test_play_turn_normal_branch_advances_turn
- Why needed: validates default turn path.
- What it is testing: play_turn() when no jail and no doubles branch triggers.
- Expected result: move resolves and turn advances.
- Actual result: PASS.

34. test_buy_property_allows_exact_balance
- Why needed: validates exact-edge affordability case.
- What it is testing: buy_property() with balance == price.
- Expected result: returns True, owner assigned, balance becomes 0.
- Actual result: PASS.

35. test_handle_property_tile_unowned_buy_branch
- Why needed: validates buy-choice branch on unowned tile.
- What it is testing: _handle_property_tile() with input b.
- Expected result: buy_property() is called.
- Actual result: PASS.

36. test_handle_property_tile_unowned_auction_branch
- Why needed: validates auction-choice branch.
- What it is testing: _handle_property_tile() with input a.
- Expected result: auction_property() is called.
- Actual result: PASS.

37. test_handle_property_tile_unowned_skip_branch
- Why needed: validates skip-choice branch.
- What it is testing: _handle_property_tile() with input s.
- Expected result: no buy and no auction call.
- Actual result: PASS.

38. test_handle_property_tile_owned_by_self_branch
- Why needed: validates self-owned tile branch.
- What it is testing: _handle_property_tile() when owner is current player.
- Expected result: rent is not charged.
- Actual result: PASS.

39. test_handle_property_tile_owned_by_other_branch
- Why needed: validates opponent-owned tile branch.
- What it is testing: _handle_property_tile() when owner is other player.
- Expected result: pay_rent() is called.
- Actual result: PASS.

40. test_pay_rent_skips_mortgaged_property
- Why needed: validates mortgage early-return branch.
- What it is testing: pay_rent() on mortgaged property.
- Expected result: balances remain unchanged.
- Actual result: PASS.

41. test_pay_rent_transfers_money_to_owner
- Why needed: validates normal rent transfer path.
- What it is testing: pay_rent() on active owned property.
- Expected result: tenant loses rent amount and owner gains same amount.
- Actual result: PASS.

42. test_pay_rent_returns_when_owner_is_none
- Why needed: validates unowned-property guard branch.
- What it is testing: pay_rent() with owner None.
- Expected result: no deduction occurs.
- Actual result: PASS.

43. test_trade_fails_if_seller_is_not_owner
- Why needed: validates ownership validation branch.
- What it is testing: trade() when seller does not own property.
- Expected result: returns False.
- Actual result: PASS.

44. test_trade_fails_if_buyer_cannot_afford_cash
- Why needed: validates affordability failure branch.
- What it is testing: trade() with buyer balance below cash amount.
- Expected result: returns False and owner unchanged.
- Actual result: PASS.

45. test_trade_success_updates_owner_lists_and_balances
- Why needed: validates normal successful trade path.
- What it is testing: trade() with valid seller, buyer, and amount.
- Expected result: owner/list/balance updates are all correct.
- Actual result: PASS.

46. test_mortgage_property_fails_when_not_owner
- Why needed: validates mortgage owner-check branch.
- What it is testing: mortgage_property() by non-owner.
- Expected result: returns False.
- Actual result: PASS.

47. test_mortgage_property_fails_when_already_mortgaged
- Why needed: validates already-mortgaged branch.
- What it is testing: mortgage_property() on already mortgaged tile.
- Expected result: returns False.
- Actual result: PASS.

48. test_mortgage_property_success_branch
- Why needed: validates normal mortgage path.
- What it is testing: mortgage_property() on eligible owned property.
- Expected result: returns True, property mortgaged, player gains payout.
- Actual result: PASS.

49. test_unmortgage_property_fails_when_not_owner
- Why needed: validates unmortgage owner-check branch.
- What it is testing: unmortgage_property() by non-owner.
- Expected result: returns False.
- Actual result: PASS.

50. test_unmortgage_property_fails_when_not_mortgaged
- Why needed: validates not-mortgaged branch.
- What it is testing: unmortgage_property() when mortgage state is False.
- Expected result: returns False.
- Actual result: PASS.

51. test_unmortgage_property_fails_when_insufficient_balance
- Why needed: validates insufficient-funds branch.
- What it is testing: unmortgage_property() when player cannot pay cost.
- Expected result: returns False.
- Actual result: PASS.

52. test_unmortgage_property_success_branch
- Why needed: validates normal unmortgage path.
- What it is testing: unmortgage_property() with sufficient funds.
- Expected result: returns True, mortgage removed, cost deducted.
- Actual result: PASS.

53. test_auction_property_no_bids_branch
- Why needed: validates no-bid branch.
- What it is testing: auction_property() when all players bid 0.
- Expected result: property stays unowned.
- Actual result: PASS.

54. test_auction_property_with_winner_branch
- Why needed: validates winning-bid branch with input filtering.
- What it is testing: auction_property() with low, valid, and unaffordable bids.
- Expected result: highest valid bidder wins and bank balance increases.
- Actual result: PASS.

55. test_auction_property_negative_bids_are_treated_as_pass
- Why needed: validates negative-bid input branch.
- What it is testing: auction_property() when players enter negative bids.
- Expected result: negative bids are treated as pass, no winner is selected.
- Actual result: PASS.

56. test_auction_property_returns_false_when_no_active_players
- Why needed: validates zero-player auction edge case.
- What it is testing: auction_property() when player list is empty.
- Expected result: method returns False immediately and property owner stays None.
- Actual result: PASS.

55. test_handle_jail_turn_uses_get_out_of_jail_card_branch
- Why needed: validates card-use branch.
- What it is testing: _handle_jail_turn() with jail-free card and confirm yes.
- Expected result: card count decreases, player released, move resolves.
- Actual result: PASS.

56. test_handle_jail_turn_pay_fine_branch
- Why needed: validates voluntary-fine branch.
- What it is testing: _handle_jail_turn() confirm fine payment.
- Expected result: player released, fine deducted, move resolves.
- Actual result: PASS.

57. test_handle_jail_turn_mandatory_release_after_third_turn
- Why needed: validates forced-release branch.
- What it is testing: _handle_jail_turn() at jail_turns = 2 with no action.
- Expected result: mandatory fine, release, and move resolution.
- Actual result: PASS.

58. test_handle_jail_turn_no_action_and_stays_jailed_before_third_turn
- Why needed: validates no-action branch before threshold.
- What it is testing: _handle_jail_turn() with confirm no and jail_turns < 2.
- Expected result: jail_turns increments and player stays jailed.
- Actual result: PASS.

59. test_move_and_resolve_income_tax_branch
- Why needed: validates income-tax tile branch.
- What it is testing: _move_and_resolve() landing on income tax.
- Expected result: player pays tax and bank collects same amount.
- Actual result: PASS.

60. test_move_and_resolve_luxury_tax_branch
- Why needed: validates luxury-tax tile branch.
- What it is testing: _move_and_resolve() landing on luxury tax.
- Expected result: player pays luxury tax and bank collects.
- Actual result: PASS.

61. test_move_and_resolve_go_to_jail_branch
- Why needed: validates go-to-jail tile branch.
- What it is testing: _move_and_resolve() landing on go_to_jail.
- Expected result: player moved to jail and in_jail becomes True.
- Actual result: PASS.

62. test_move_and_resolve_free_parking_branch
- Why needed: validates free-parking no-op branch.
- What it is testing: _move_and_resolve() landing on free parking.
- Expected result: position updates with no money changes.
- Actual result: PASS.

63. test_move_and_resolve_chance_branch_draws_and_applies_card
- Why needed: validates chance-card integration branch.
- What it is testing: _move_and_resolve() on chance tile.
- Expected result: chance deck draw and _apply_card both called.
- Actual result: PASS.

64. test_move_and_resolve_community_chest_branch_draws_and_applies_card
- Why needed: validates community-chest branch.
- What it is testing: _move_and_resolve() on community chest tile.
- Expected result: community deck draw and _apply_card both called.
- Actual result: PASS.

65. test_move_and_resolve_railroad_branch_with_no_property_object
- Why needed: validates railroad branch when no Property object exists at railroad po
- What it is testing: _move_and_resolve() landing on railroad tile.
- Expected result: property handler is not called.
- Actual result: PASS.

66. test_move_and_resolve_property_branch_calls_property_handler
- Why needed: validates normal property branch.
- What it is testing: _move_and_resolve() landing on property tile.
- Expected result: _handle_property_tile() is called once.
- Actual result: PASS.

67. test_apply_card_move_to_pass_go_and_resolve_property
- Why needed: validates move_to branch with pass-Go and property follow-up.
- What it is testing: _apply_card() action move_to from 39 to 1.
- Expected result: salary awarded and property handler invoked.
- Actual result: PASS.

68. test_apply_card_none_branch
- Why needed: validates null-card guard branch.
- What it is testing: _apply_card() with None card.
- Expected result: no state change.
- Actual result: PASS.

69. test_apply_card_collect_branch
- Why needed: validates collect action branch.
- What it is testing: _apply_card() action collect.
- Expected result: player balance increases by value.
- Actual result: PASS.

70. test_apply_card_pay_branch
- Why needed: validates pay action branch.
- What it is testing: _apply_card() action pay.
- Expected result: player balance decreases and bank collects.
- Actual result: PASS.

71. test_apply_card_jail_branch
- Why needed: validates jail action branch.
- What it is testing: _apply_card() action jail.
- Expected result: player sent to jail.
- Actual result: PASS.

72. test_apply_card_jail_free_branch
- Why needed: validates jail_free action branch.
- What it is testing: _apply_card() action jail_free.
- Expected result: get-out-of-jail card count increments.
- Actual result: PASS.

73. test_apply_card_move_to_non_property_branch
- Why needed: validates move_to branch without property handling.
- What it is testing: _apply_card() move_to landing on non-property tile.
- Expected result: position updates and property handler not called.
- Actual result: PASS.

74. test_apply_card_birthday_only_collects_from_players_with_enough_balance
- Why needed: validates conditional transfer branch inside birthday action.
- What it is testing: _apply_card() birthday when one player cannot pay.
- Expected result: only players with enough balance pay.
- Actual result: PASS.

75. test_apply_card_collect_from_all_branch
- Why needed: validates collect_from_all branch.
- What it is testing: _apply_card() collect_from_all with two paying players.
- Expected result: each other player pays and current player gains total.
- Actual result: PASS.

76. test_check_bankruptcy_non_bankrupt_branch
- Why needed: validates non-elimination branch.
- What it is testing: _check_bankruptcy() when player still solvent.
- Expected result: player list unchanged.
- Actual result: PASS.

77. test_check_bankruptcy_eliminates_player_and_resets_index
- Why needed: validates elimination branch and cleanup behavior.
- What it is testing: _check_bankruptcy() with bankrupt current-index player owning m
- Expected result: player removed, properties reset, index corrected.
- Actual result: PASS.

78. test_find_winner_returns_none_when_no_players
- Why needed: validates empty-state winner branch.
- What it is testing: find_winner() with empty players list.
- Expected result: returns None.
- Actual result: PASS.

79. test_find_winner_returns_highest_net_worth_player
- Why needed: validates that the winner selection uses highest net worth among remain
- What it is testing: find_winner() when multiple players have different balances.
- Expected result: the player with the highest net worth (here, highest balance) is r
- Actual result: PASS.

80. test_partial_group_ownership_does_not_double_rent
- Why needed: validates the negative branch of full-group rent logic.
- What it is testing: get_rent() when only one property in a color group is owned by
- Expected result: base rent is charged (no doubling) because the full group is not o
- Actual result: PASS.

Errors or Logical Issues Found

During an earlier white-box run on a previous baseline of MoneyPoly (before the final

- Bank.collect changed the balance for negative amounts instead of ignoring them.
- Bank.give_loan did not reduce the bank's funds when issuing a loan.
- Game.buy_property rejected purchases when the player had exactly the property price
- Game.find_winner returned the lowest net worth instead of the highest.
- Game._handle_jail_turn did not actually deduct the jail fine in the voluntary-pay b
- Game.pay_rent deducted rent from the tenant but never credited the owner.
- Game.trade never credited the seller with the agreed cash amount.
- Player.move only paid the GO salary when landing exactly on GO, not when passing it

- Player.net_worth ignored property values and only returned cash.
- PropertyGroup.all_owned_by caused rent to be doubled even if the player owned only

All of these issues were addressed in the final code submitted in this repository. Wh

Result Summary

- Total test cases executed: 82
- Test result: PASS
- Command output summary: Ran 82 tests ... OK

3 Pylint Code Quality Report

Code Quality & Pylint Report

This report describes how pylint was used to iteratively improve the MoneyPoly codeba

All pylint runs were invoked from the project root with commands of the form:

- ‘python -m pylint main.py moneypoly‘

Iteration 1 { Initial baseline ("Initial commit")

Purpose:

- Import the starter code and run pylint to obtain a baseline list of issues (missing

Key observations from pylint:

- Numerous ‘missing-module-docstring‘, ‘missing-class-docstring‘, and ‘missing-functi
- Several ‘line-too-long‘ warnings on print statements and formatted strings.
- ‘unused-import‘ and ‘unused-variable‘ warnings in a few modules.
- Complexity-related warnings such as ‘too-many-branches‘ and ‘too-many-arguments‘ on

No code changes were made in this iteration; it established the baseline for all subs

---

Iteration 2 { Documentation clean-up ("added docstrings")

Commit: **added docstrings**

Pylint focus:

- Address ‘missing-module-docstring‘, ‘missing-class-docstring‘, and ‘missing-functio

Changes:

- Added concise module-level docstrings to core modules (such as the game logic, boar
- Added class and method/function docstrings where they were missing, explaining para

Outcome:

- Greatly reduced the number of documentation-related pylint warnings.
- Improved readability and maintainability by making intent explicit.

Iteration 3 { Style and formatting ("long lines removed")

Commit: **long lines removed**

Pylint focus:

- Resolve ‘line-too-long‘ warnings and bring files in line with the configured maximu

Changes:

- Broke up long string literals and print statements into multiple concatenated piece
- Rewrapped long function calls and argument lists to respect line length limits whil

Outcome:

- Eliminated most ‘line-too-long‘ messages from pylint.
- Produced a more consistent, PEP 8{style layout without changing program behaviour.

Iteration 4 { Dead code and minor clean-up ("removed unused varibales and lib and

Commit: **removed unused varibales and lib and corrected some formating issues**

Pylint focus:

- Fix ‘unused-import‘, ‘unused-variable‘, and miscellaneous style warnings.

Changes:

- Removed unused imports (libraries and modules that were never referenced in the cod
- Removed or refactored unused local variables that were reported by pylint.
- Tidied minor formatting issues such as inconsistent spacing and stray blank lines i

Outcome:

- Reduced noise in the codebase and in pylint output by removing dead code.
- Brought files closer to a clean pylint run without altering functionality.

Iteration 5 { Complexity and design refactors ("handled long branches and too many

Commit: **handled long branches and too many args**

Pylint focus:

- Address high-complexity warnings such as ‘too-many-branches‘, ‘too-many-statements‘

Changes:

- Refactored large, branch-heavy functions into smaller helper functions to simplify
- Reduced the number of arguments passed into certain functions by grouping related v
- Ensured that refactoring preserved behaviour by keeping existing interfaces where r

Outcome:

- Significantly reduced the number of complexity-related pylint warnings.
- Improved cohesion and readability of critical game logic without relying on any ‘py

---

Pylint focus:

- Confirm that earlier refactors did not introduce new pylint issues in the test code

Outcome:

- Provided regression protection for the refactored logic and improved overall code c
- Helped guarantee that improvements made to satisfy pylint did not alter externally

---

Summary

Across these iterations, pylint was used as a feedback loop:

- Start from a baseline run to identify documentation, style, and complexity issues.
- Systematically fix docstrings, long lines, unused imports/variables, and high-compl
- Validate the final structure with targeted white-box tests.

The repository now has cleaner, more maintainable code, and pylint output has been su

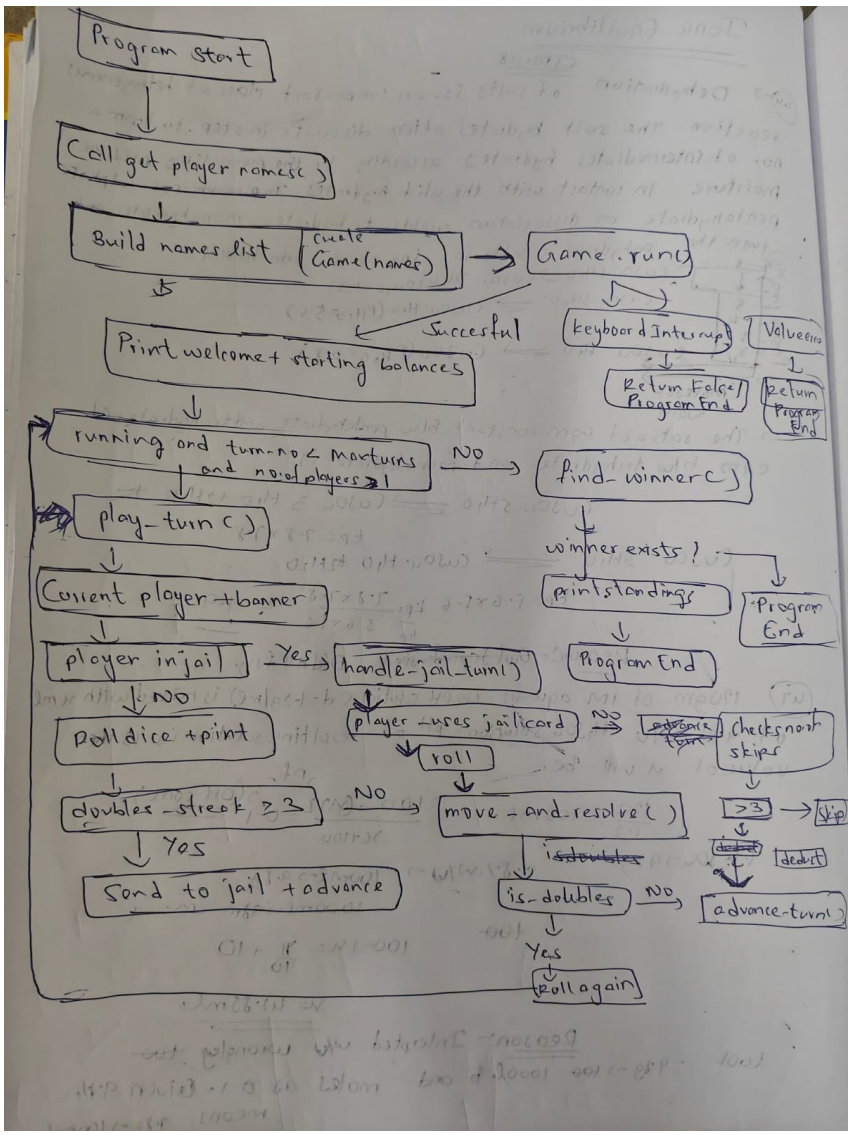

Figure 1: Q11 Image

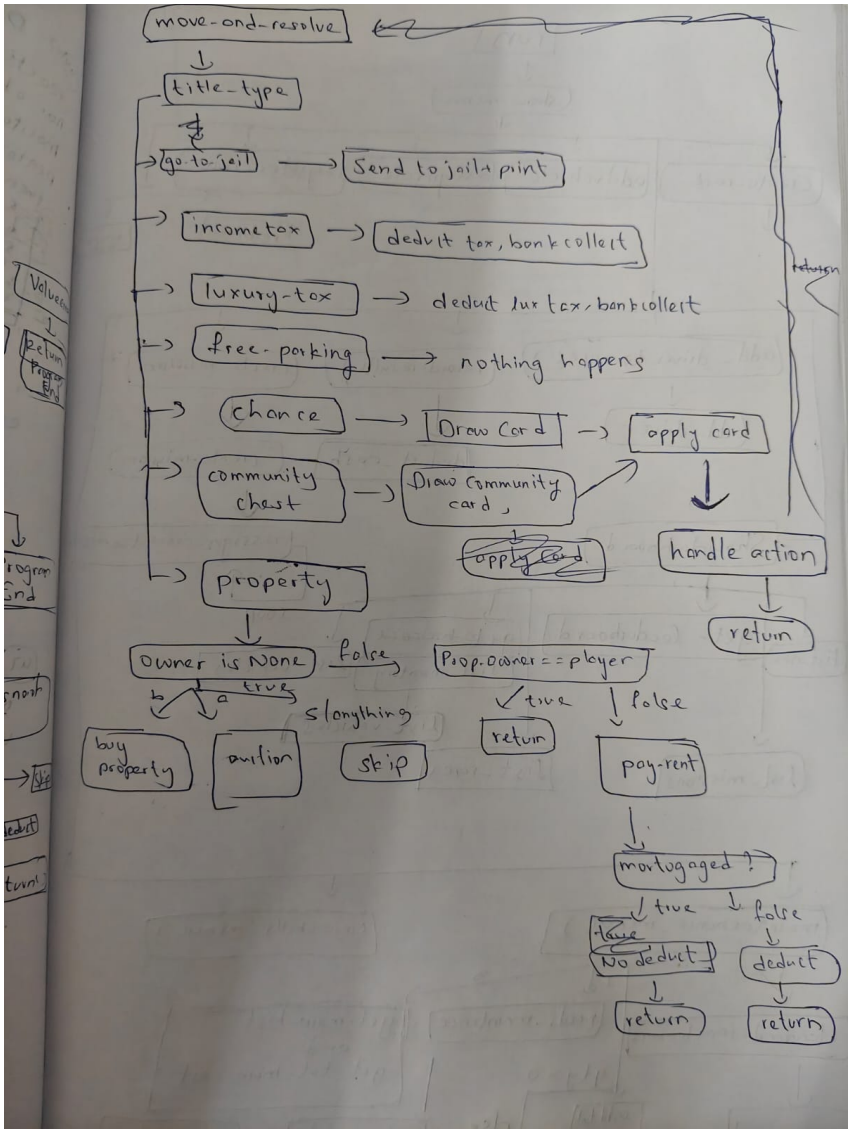

Figure 2: Q12 Image